Relatório Técnico — Análise do Hipervisor e do Impacto da Máquina Virtual sobre o Host

Disciplina: Arquitetura de Sistemas Computacionais
Programa: PPGIA — Mestrado em Informática — PUC-PR
Integrantes: Fernando Sabino Dantas, Luciano Rasera **Data:** 15 de abril de 2026

https://github.com/fsd-dantas/ppgia-puc-pr-asc

---

1. Caracterização do Ambiente

1.1 Sistema Hospedeiro (Host)

Atributo	Valor
Sistema Operacional	Ubuntu 24.04.4 LTS (kernel 6.8.0-106-generic)
Processador	Intel Core i7-3770 @ 3.40 GHz
Memória RAM	16 GB
Armazenamento	3× HDD SATA 465.8 GB (`sda`, `sdb`, `sdc`)
Núcleos disponíveis	4 cores / 8 threads (HT habilitado)
Suporte a virtualização	VT-x confirmado (`grep -c vmx /proc/cpuinfo` → 16)
Software de virtualização	KVM/QEMU com libvirt
Hardware	Dell OptiPlex 9010 (Desktop)

Notas sobre o armazenamento:

- `sda`: disco do sistema operacional host (LVM — `ubuntu-vg`, 100 GB alocados, 86 GB livres no FS)
- `sdb`: disco dedicado ao disco virtual da VM (`/dev/sdb2`, raw block device, sem filesystem)
- `sdc`: disco adicional com partição ext4 não montada

1.2 Máquina Virtual (Guest)

Atributo	Valor
Sistema Operacional convidado	Ubuntu 24.04.4 LTS Server (kernel 6.8.0-107-generic x86_64)
Hostname	`cisei-guestvm-01`
vCPUs atribuídas	4
Memória RAM atribuída	4 GB (uso em idle: ~7%)
Swap	0 B (desabilitado)
Disco virtual	`/dev/sdb2` (raw block device) — filesystem 97.87 GB, 6.6% ocupado

Atributo	Valor
Tipo de disco virtual	Raw block device passado diretamente via virtio
Rede	`enp1s0`, driver virtio, IP 192.168.122.18
Processos em idle	156
Outras configurações	VT-x habilitado no host; aceleração KVM ativa

Ferramentas de monitoramento confirmadas no guest:

Ferramenta	Caminho	Pacote
`iostat`	`/usr/bin/iostat`	`sysstat`
`vmstat`	`/usr/bin/vmstat`	`procps`
`strace`	`/usr/bin/strace`	`strace`
`python3`	`/usr/bin/python3`	`python3`
`iotop`	`/usr/sbin/iotop`	`iotop`

---

2. Experimentos

> Experimento 1: **10 repetições**. Experimentos 2 e 3: **3 repetições** com limpeza de cache entre runs. Os valores apresentados são a mediana das execuções.

Experimento 1 — Carga de CPU dentro da VM

Objetivo: Executar uma tarefa computacional intensiva e comparar o consumo de CPU no guest e no processo da VM no host.

Comandos executados no guest (10 repetições):

```
for i in $(seq 1 10); do
  echo -n "run $i: "
  { time python3 -c "sum(i*i for i in range(10**7))"; } 2>&1 | grep real
done | tee dados/exp1-cpu/tempo-guest.txt
```

Monitoramento no host (executado antes do loop acima):

```
pidstat -p 2365 1 30 | tee dados/exp1-cpu/pidstat-host.txt
```

Resultados — tempos no guest:

Run	Tempo real (s)
1	0.889

Run	Tempo real (s)
2	0.841
3	0.835
4	0.883
5	0.878
6	0.834
7	0.836
8	0.881
9	0.880
10	0.827
Mediana	**0.860**
Mínimo	0.827
Máximo	0.889
Média	0.858

Resultados — processo QEMU no host (`pidstat`, intervalo 12:32:51–12:32:59):

Métrica	Idle (antes)	Pico (durante burst)	Observações
`%guest`	1%	101–102%	vCPU executando código do guest via KVM
`%system`	0%	0%	Sem VM-exits durante carga pura de CPU
`%usr`	0%	0–1%	QEMU não usa CPU user-space para execução do guest
`%CPU`	0%	101%	Total consumido pelo processo QEMU no host
CPU (core)	1	1	vCPU fixada no core 1 durante todo o experimento

> `%guest > 100%` é artefato de arredondamento do `pidstat` quando o processo usa múltiplas vCPUs dentro de um único intervalo de amostragem.

Interpretação: O campo `%guest` representa o tempo em que o núcleo físico executou diretamente código do guest via aceleração KVM — sem interposição do QEMU em modo usuário. A ausência de `%system` durante o burst confirma que carga puramente computacional (sem I/O ou instruções privilegiadas) não gera VM-exits, caracterizando execução nativa mediada pelo hardware (VT-x).

Saída bruta: `dados/exp1-cpu/`

Experimento 2 — Escrita em disco dentro da VM (obrigatório)

Objetivo: Observar como a escrita no disco virtual se manifesta como atividade de I/O no host.

Comandos executados no guest (3 repetições com limpeza de cache):

```
iostat -xz vda 1 120 > dados/exp2-disco-escrita/iostat-guest.txt &
for i in $(seq 1 3); do
  echo "=== run $i ===" | tee -a dados/exp2-disco-escrita/tempo-guest.txt
  echo 3 | sudo tee /proc/sys/vm/drop_caches > /dev/null
  { time dd if=/dev/zero of=teste.bin bs=1M count=512 conv=fdatasync; } 2>&1 | tee
-a dados/exp2-disco-escrita/tempo-guest.txt
  sleep 5
done
```

Monitoramento no host (executado antes do loop acima):

```
pidstat -d -p 2365 1 120 > dados/exp2-disco-escrita/pidstat-host.txt &
iostat -xz sdb 1 120 > dados/exp2-disco-escrita/iostat-host.txt &
```

Resultados — tempos e throughput no guest:

Run	Tempo dd (s)	Throughput (MB/s)	Tempo real (s)	sys (s)
1	5.819	92.3	6.082	0.562
2	5.684	94.4	5.937	0.459
3	5.786	92.8	5.983	0.422
Mediana	**5.786**	**92.8**	**5.983**	**0.459**

> `real - dd` ≈ 0.2–0.3 s por run = overhead do `fdatasync` final (flush forçado até o disco físico). `sys` (~8% do tempo real) = CPU do kernel guest em syscalls de escrita; restante = CPU bloqueada aguardando I/O.

Resultados — atividade no host (`iostat -xz sdb`):

Métrica	Idle (antes)	Escrita ativa	Pico	Observações
`wkB/s`	14	95.000–103.000	106.704	Virtio agrega writes do guest
`wareq-sz` (kB)	35	642–1133	1133	Coalescing virtio → requisições grandes
`w_await` (ms)	62	197–715	715	Latência HDD + fila de escrita
`%iowait`	0.27%	5–12%	12.42%	Host CPU aguardando disco
`%util` sdb	0.72%	95–99%	99.5%	Disco praticamente saturado
`aqu-sz`	0.03	30–65	65	Fila profunda: disco é o gargalo

> O host enxerga throughput ligeiramente **maior** que o guest reporta (até ~104 MB/s vs. 92–94 MB/s) porque o `virtio-blk` agrega múltiplas requisições menores do guest em operações de bloco maiores para o host, aumentando a eficiência de transferência.

Saída bruta: `dados/exp2-disco-escrita/`

Experimento 3 — Leitura em disco dentro da VM

Objetivo: Comparar o comportamento de leitura em relação à escrita; avaliar efeito de cache.

Pré-requisito: arquivo `teste.bin` criado no Experimento 2.

Comandos executados no guest (3 repetições com limpeza de cache):

```
for i in $(seq 1 3); do
  echo "=== run $i ===" | tee -a dados/exp3-disco-leitura/tempo-guest.txt
  echo 3 | sudo tee /proc/sys/vm/drop_caches > /dev/null
  { time dd if=teste.bin of=/dev/null bs=1M; } 2>&1 | tee -a dados/exp3-disco-
leitura/tempo-guest.txt
  sleep 3
done
```

Monitoramento no host (executado antes do loop acima):

```
pidstat -d -p 2365 1 60 > dados/exp3-disco-leitura/pidstat-host.txt &
iostat -xz sdb 1 60 > dados/exp3-disco-leitura/iostat-host.txt &
```

Resultados — tempos e throughput no guest:

Run	Tempo dd (s)	Throughput (MB/s)	Tempo real (s)	sys (s)
1	5.660	94.8	5.849	0.941
2	5.400	99.4	5.598	0.393
3	5.422	99.0	5.620	0.395
Mediana	**5.422**	**99.0**	**5.620**	**0.395**

> Run 1 apresentou `sys=0.941s` (2.4× maior que as demais) — provável overhead de reposicionamento do cabeçote HDD após a sequência de escritas do Experimento 2. Runs 2 e 3, com disco já posicionado sequencialmente, foram estáveis.

Resultados — atividade no host (`iostat -xz sdb`):

Métrica	Idle (antes)	Leitura ativa	Pico	Observações
`rkB/s`	1	93.000–103.000	103.424	Leitura sequencial via virtio

Métrica	Idle (antes)	Leitura ativa	Pico	Observações
`rareq-sz` (kB)	36	428–1024	1024	Grandes blocos sequenciais
`r_await` (ms)	6	17–31	31	Latência de leitura muito menor que escrita
`%iowait`	0.27%	0–5%	5.03%	Metade do %iowait da escrita
`%util` sdb	0.72%	77–91%	91%	Menor saturação que escrita
`aqu-sz`	0.03	1.8–5.2	5.2	Fila ~10× menor que na escrita

> `pidstat -d` retornou `-1` para todos os campos — o QEMU acessa `/dev/sdb2` como raw block device via KVM, o que bypassa a contabilidade por-processo de I/O do `/proc/<pid>/io`. O kernel registra a atividade no nível do dispositivo (visível em `iostat`), mas não a associa ao processo QEMU. Isso é em si uma evidência do modelo de passagem direta de dispositivo no KVM.

Saída bruta: `dados/exp3-disco-leitura/`

Experimento 4 — Observação de chamadas de sistema

Objetivo: Identificar as syscalls geradas por operações simples no guest e discutir a mediação pelo hipervisor.

Comandos executados no guest:

```
strace -c ls > /dev/null 2> dados/exp4-syscalls/strace-ls.txt
strace -c cat teste.bin > /dev/null 2> dados/exp4-syscalls/strace-cat.txt
```

Resultados — `strace -c ls`:

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 32.72    0.000655          36        18           mmap
 15.98    0.000320          45         7           openat
  8.04    0.000161          32         5           mprotect
  7.14    0.000143          28         5           read
  6.79    0.000136          15         9           close
  6.54    0.000131          65         2         2 access
  4.95    0.000099          49         2           getdents64
  4.75    0.000095          11         8           fstat
  3.25    0.000065          32         2         2 statfs
  2.05    0.000041          41         1           munmap
  1.50    0.000030          10         3           brk
  1.35    0.000027          13         2         2 ioctl
  1.35    0.000027          13         2           pread64
  0.95    0.000019          19         1           getrandom
  0.55    0.000011          11         1           write
  0.45    0.000009           9         1           arch_prctl
  0.45    0.000009           9         1           prlimit64
  0.45    0.000009           9         1           rseq
```

```
  0.40    0.000008           8         1           set_tid_address
  0.35    0.000007           7         1           set_robust_list
  0.00    0.000000           0         1           execve
------ ----------- ----------- --------- --------- ----------------
100.00    0.002002          27        74         6 total
```

Resultados — `strace -c cat teste.bin`:

```
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 84.35    0.215639          52      4098           read
 14.84    0.037940           9      4096           write
  0.26    0.000672         672         1           execve
  0.19    0.000489          48        10           mmap
  0.08    0.000207          51         4           openat
  0.05    0.000133          44         3           mprotect
  0.05    0.000123          61         2           munmap
  0.04    0.000098          32         3           brk
  0.03    0.000085          17         5           fstat
  0.03    0.000067          11         6           close
  0.02    0.000054          27         2           pread64
  0.01    0.000034          34         1           arch_prctl
  0.01    0.000031          31         1         1 access
  0.01    0.000017          17         1           getrandom
  0.01    0.000015          15         1           fadvise64
  0.01    0.000015          15         1           prlimit64
  0.01    0.000014          14         1           set_tid_address
  0.01    0.000014          14         1           rseq
  0.01    0.000013          13         1           set_robust_list
------ ----------- ----------- --------- --------- ----------------
100.00    0.255660          31      8238         1 total
```

Observações:

Aspecto	`ls`	`cat teste.bin`
Total de chamadas	74	8.238
Tempo total (strace)	2 ms	256 ms
Syscall dominante	`mmap` (32.7%) — carregamento de libs	`read` (84.4%) — I/O de dados
Chamadas de I/O de dados	`getdents64` ×2 (listagem do diretório)	`read` ×4098 + `write` ×4096
Tamanho de buffer inferido	—	512 MB / 4096 reads = **128 KB/chamada**
Overhead do `strace`	—	cat sem strace: ~5.4 s; com strace: 256 ms em syscall-time (serialização)

> No `ls`, 97% das chamadas são inicialização do processo (dynamic linker: `mmap`, `openat`, `mprotect`). O trabalho real — listar o diretório — é apenas `getdents64` ×2. No `cat`, 99.5% das chamadas são `read`/`write` em loop com buffer de 128 KB, diretamente medíveis como trabalho de I/O.

Saída bruta: `dados/exp4-syscalls/`

---

3. Monitoramento Simultâneo

3.1 Ferramentas utilizadas

Ferramenta	Contexto	Finalidade
`top` / `htop`	Guest e Host	Uso de CPU e memória em tempo real
`iostat -xz 1`	Guest e Host	Atividade de disco, %iowait, throughput
`pidstat`	Host	CPU/I/O do processo específico da VM
`vmstat 1`	Guest	Memória, swap, I/O por bloco, CPU
`iotop`	Host	Processos com maior atividade de I/O
`strace -c`	Guest	Contagem de chamadas de sistema
`time`	Guest	Medição de tempo de execução
`/proc/<pid>/status`	Guest e Host	Estado e consumo de memória do processo

3.2 Processo da VM no Host

```
# Identificação do processo da VM
ps -eo pid,ppid,comm,%cpu,%mem --sort=-%cpu | grep qemu
```

Processo identificado: `qemu-system-x86_64` (VM ID 2 — `guest-ubuntu2404`, estado `running`)

> O PID exato varia a cada inicialização da VM. Para capturar o PID em tempo de execução:
>
> ```
> virsh dominfo guest-ubuntu2404   # confirma ID e estado
> ps -eo pid,comm | grep qemu-system
> ```
>
> Anotar o PID e substituir `<PID_VM>` nos comandos de monitoramento (`pidstat -p <PID_VM> 1`).

---

4. Resultados Consolidados

Experimento	Métrica observada	Valor no Guest	Valor no Host	Observações
1 — CPU	Tempo real — mediana (s)	0.860	—	10 repetições
1 — CPU	CPU pico	~97% (`user`/`real`)	`%guest`=101%, `%CPU`=101%	Execução via KVM/VT-x
1 — CPU	`%system` no host (pico)	~2%	0%	Sem VM-exits para carga pura de CPU
1 — CPU	`%usr` do processo QEMU	N/A	0%	QEMU não usa CPU user-space para exec. do guest
2 — Escrita	Tempo real — mediana (s)	5.983	—	3 repetições com drop_caches
2 — Escrita	Throughput — mediana (MB/s)	92.8	~104 (pico)	Host enxerga mais: virtio-blk coalescing
2 — Escrita	`%iowait` — pico	—	12.42%	Escrita síncrona (fdatasync) bloqueia CPU
2 — Escrita	`%util` sdb	—	95–99%	Disco praticamente saturado
2 — Escrita	`w_await` — faixa (ms)	—	144–715	Alta latência: flush forçado + fila profunda
2 — Escrita	`aqu-sz` — pico	—	65	Fila de escrita profunda
3 — Leitura	Tempo real — mediana (s)	5.620	—	3 repetições com drop_caches
3 — Leitura	Throughput — mediana (MB/s)	99.0	~101 (pico)	+7% vs escrita; sem overhead de sync
3 — Leitura	`%iowait` — pico	—	5.03%	Metade do %iowait da escrita
3 — Leitura	`%util` sdb	—	77–91%	Menor saturação que escrita
3 — Leitura	`r_await` — faixa (ms)	—	17–31	5–20× menor que `w_await`
3 — Leitura	`aqu-sz` — pico	—	5.2	Fila ~10× menor que na escrita
4 — Syscalls	Total de chamadas (`ls`)	74 (2 ms)	N/A	97% inicialização de processo (dynamic linker)
4 — Syscalls	Total de chamadas (`cat`)	8.238 (256 ms)	N/A	99.5% read/write; 128 KB/chamada

5. Análise e Discussão

5.1 Impacto do host por cargas executadas na VM

Os experimentos demonstraram correlação direta entre a carga aplicada no guest e o consumo de recursos observado no host, com características distintas conforme o tipo de carga.

Na carga de CPU (Experimento 1), o processo `qemu-system-x86_64` no host atingiu `%CPU=101%` durante a execução do script Python, contra uma linha de base de 0.36% em idle. O aumento foi imediato e proporcional: enquanto o guest reportou ~97% de CPU user-space, o host registrou `%guest=101%` no mesmo intervalo de amostragem. Após o término do script, o processo retornou ao baseline em menos de 1 segundo.

Nas cargas de disco (Experimentos 2 e 3), o disco `sdb` saiu de `%util=0.72%` (idle) para 95–99% durante escritas e 77–91% durante leituras, com `rkB/s`/`wkB/s` passando de 14 kB/s para valores entre 75.000 e 106.000 kB/s. O `%iowait` do host chegou a 12.42% nas escritas, evidenciando que a carga de I/O do guest impacta diretamente o escalonador de CPU do host ao bloquear threads aguardando o disco físico.

5.2 Comportamento do processo da VM durante os testes

O processo `qemu-system-x86_64` (PID 2365) apresentou comportamento distinto para cada tipo de carga, revelado pelo `pidstat`:

Carga de CPU: `%guest` atingiu 101%, enquanto `%usr` permaneceu em 0% e `%system` em 0%. Esse padrão indica que a CPU guest executa diretamente no hardware via KVM, sem interposição do QEMU em modo usuário. O campo `%guest` é específico do `pidstat` para processos KVM e representa o tempo em que o núcleo físico executou em modo guest (VMX non-root operation).

Carga de disco (escrita/leitura): O `pidstat -d` retornou `-1` para todos os campos de I/O (`kB_rd/s`, `kB_wr/s`). Isso ocorre porque o QEMU acessa `/dev/sdb2` como raw block device via passagem direta ao KVM — o kernel não contabiliza esse I/O em `/proc/<pid>/io`, rastreando-o apenas no nível do dispositivo de bloco. A ausência de métricas por processo é, em si, uma consequência da arquitetura de passagem direta de dispositivo.

5.3 Escrita no guest vs. atividade perceptível no host

A escrita no guest gerou atividade de I/O claramente mensurável no host. O `wkB/s` do `sdb` saltou de 14 kB/s (baseline) para picos de 106.704 kB/s (~104 MB/s) durante o `dd conv=fdatasync` no guest.

O `fdatasync` ao final de cada `dd` forçou o flush completo da pilha de cache até o disco físico, tornando visível no host o padrão de "acumulação + descarga": durante a escrita principal, o `w_await` ficou entre 144 e 400 ms (fila crescente), chegando a 715 ms no momento do sync final, quando o buffer do disco precisou ser esvaziado para o meio magnético.

O `wareq-sz` passou de 35 kB (idle) para 642–1133 kB durante as escritas. Isso reflete o comportamento do driver `virtio-blk`: em vez de repassar cada escrita do guest individualmente ao bloco físico, ele agrega múltiplas requisições em operações maiores, reduzindo overhead de IOPS e explicando por que o host registrou throughput ligeiramente superior ao reportado pelo guest.

5.4 Diferença entre leitura e escrita

A leitura apresentou desempenho superior em todas as métricas relevantes:

Métrica	Escrita	Leitura	Fator
Throughput guest (mediana)	92.8 MB/s	99.0 MB/s	+6.7%
`await` no host	144–715 ms	17–31 ms	5–23× menor
`%iowait` pico	12.42%	5.03%	2.5× menor
`aqu-sz` pico	65	5.2	12.5× menor
`%util` sdb	95–99%	77–91%	Menor saturação

As diferenças têm duas causas principais. Primeiro, o `conv=fdatasync` nas escritas impõe uma barreira de durabilidade: o `dd` só retorna após os dados atingirem o disco físico, acumulando latência de seek, rotação e escrita magnética. Nas leituras, não há equivalente — o `dd` retorna assim que os dados chegam ao buffer do kernel. Segundo, HDDs têm latência de leitura intrinsecamente menor que de escrita sequencial porque o cabeçote pode iniciar a leitura ao encontrar os dados no prato, enquanto a escrita exige localização de setor livre, escrita e verificação.

A fila profunda de escrita (`aqu-sz` até 65) versus a fila rasa de leitura (`aqu-sz` até 5.2) reflete diretamente esse comportamento: o disco estava saturado acumulando escritas pendentes, enquanto as leituras sequenciais foram absorvidas com menor backpressure.

5.5 Dependência da camada de virtualização

O desempenho observado depende tanto do sistema convidado quanto da camada de virtualização, mas de formas distintas por tipo de carga.

Para CPU: a virtualização assistida por hardware (VT-x) praticamente elimina o overhead do hipervisor para código de aplicação. O `%system=0%` no processo QEMU durante a execução Python confirma que não houve VM-exits para esse workload. O tempo real no guest (0.860 s) é comparável ao que se esperaria em bare metal para a mesma operação, com a diferença de que o processo é visto pelo host como `%guest` e não como `%usr`.

Para I/O: a camada de paravirtualização (`virtio-blk`) introduz comportamento diferente do acesso direto. O coalescing de requisições pelo driver virtio alterou o `wareq-sz` observado no host (35 kB idle → 1133 kB em carga), modificando o padrão de acesso ao disco físico em relação ao que o guest requisitou. Em termos de throughput, o impacto foi neutro ou levemente positivo (host chegou a enxergar 104 MB/s contra 92.8 MB/s reportados pelo guest), mas em termos de latência de escrita o `w_await` elevado (até 715 ms) reflete o custo do flush forçado que atravessa o virtio, o driver do host e o HDD.

Diferentemente da Full Virtualization pura (sem VT-x), onde cada instrução privilegiada do guest seria interceptada e emulada por software — com overhead substancial —, o KVM com VT-x delega a execução ao hardware, reservando a intervenção do hipervisor apenas para eventos que o hardware não pode tratar autonomamente (VM-exits por I/O, interrupções, acessos a MSRs).

5.6 Evidências da atuação do hipervisor como intermediador

As seguintes saídas das ferramentas evidenciam diretamente a mediação do hipervisor:

1. **Campo `%guest` no `pidstat`** (Experimento 1): métrica exclusiva de processos KVM, representa o tempo de execução em modo VMX non-root. Valor de 101% confirma que o vCPU estava executando código do guest com aceleração de hardware — o hipervisor viabiliza essa execução, mas não a realiza em software.

2. **`%system=0%` durante carga de CPU** (Experimento 1): a ausência de `%system` no processo QEMU durante carga computacional pura confirma que não houve VM-exits para esse workload — o hardware (VT-x) tratou a execução sem intervenção do kernel host.

3. **`pidstat -d` retornando `-1`** (Experimentos 2 e 3): o QEMU acessa `/dev/sdb2` via passagem direta de raw block device ao KVM. O kernel host não consegue atribuir esse I/O ao processo QEMU — rastreando-o apenas no nível do dispositivo (`iostat sdb`). Isso evidencia que o hipervisor, ao usar passthrough de dispositivo, remove-se do caminho crítico de I/O, delegando o acesso diretamente ao driver de bloco do kernel host.

4. **`wareq-sz` de 35 kB (idle) → 642–1133 kB (escrita ativa)** (Experimento 2): a camada `virtio-blk` do hipervisor agrega requisições do guest antes de submetê-las ao disco físico. O guest requisitou blocos de 1 MB (`bs=1M` no `dd`), mas o host viu requisições de até 1133 kB — evidência de que o virtio processa e otimiza o fluxo de I/O antes de repassá-lo ao hardware.

5. **Atividade de `sdb` no host correlacionada com operações no guest** (Experimentos 2 e 3): o disco `sdb` saiu de `%util=0.72%` para 95–99% exatamente nos intervalos em que o `dd` rodava no guest, voltando ao baseline entre runs (durante os `sleep 5`). Essa correlação direta confirma que o hipervisor translada operações de disco do guest para o disco físico do host de forma transparente.

5.7 Limitações experimentais

- **Variabilidade do cache HDD:** o Run 1 do Experimento 3 apresentou `sys=0.941s` (2.4× acima dos demais), provavelmente por reposicionamento do cabeçote após a sequência de escritas. Experimentos isolados por tipo de operação reduziriam esse efeito.
- **Ausência de métricas de memória do QEMU:** o uso de memória do processo `qemu-system-x86_64` durante os experimentos não foi capturado; `/proc/2365/status` teria fornecido `VmRSS` e `VmPeak` por experimento.
- **Overhead do `strace`** (Experimento 4): o `strace` intercepta e registra cada syscall, adicionando latência por chamada. O tempo de 256 ms reportado para o `cat` com `strace` não é representativo do tempo real de execução (~5.4 s sem `strace`); o experimento mede a distribuição relativa das chamadas, não o desempenho absoluto.
- **Outros processos no host:** embora o sistema estivesse em baixa carga durante os experimentos (`%idle` > 86% na maioria dos intervalos), outros processos do sistema podem ter introduzido variância residual nas métricas de disco e CPU.

---

6. Conclusão

Os experimentos realizados neste trabalho confirmaram, com evidências mensuráveis, que o hipervisor KVM atua como intermediador entre o sistema convidado e os recursos físicos do host, com características e overhead distintos conforme o tipo de recurso virtualizado.

Para cargas de CPU, a virtualização assistida por hardware (VT-x) demonstrou eficiência próxima ao bare metal: o guest executou com ~97% de utilização de CPU enquanto o QEMU apresentou `%system=0%` e `%usr=0%` no host. O campo `%guest` do `pidstat`, específico de processos KVM, evidenciou que o vCPU executou diretamente no hardware, com o hipervisor presente apenas como habilitador da execução, e não como intermediador ativo de cada instrução.

Para cargas de disco, a camada de paravirtualização (`virtio-blk`) revelou sua natureza como intermediador ativo: o driver agregou requisições do guest em blocos maiores para o host (`wareq-sz` de 35 kB para até 1133 kB), o `fdatasync` forçou a drenagem de toda a pilha de cache até o meio magnético — gerando latências de até 715 ms e `%util` de 99% no disco —, e o `pidstat -d` retornando `-1` demonstrou que o I/O, neste modelo de passagem direta, não é contabilizado por processo, mas apenas pelo dispositivo.

A comparação entre leitura e escrita expôs o custo da durabilidade imposta pelo `fdatasync`: a leitura, sem barreiras de sincronização, atingiu throughput ~7% maior (99.0 vs 92.8 MB/s), latência 5–23× menor e fila de disco 12.5× menos profunda. O hipervisor não elimina as características físicas do disco; ele as translada fielmente para o guest, incluindo suas limitações.

O Experimento 4 complementou a análise mostrando que, no nível de syscalls, a execução no guest segue o mesmo modelo do Linux nativo: operações comuns como `ls` geram 74 chamadas majoritariamente de inicialização de processo, enquanto `cat` sobre um arquivo de 512 MB gera 8.238 chamadas com buffer de 128 KB — padrão idêntico ao esperado em bare metal. Isso confirma que a camada de virtualização é transparente para o sistema operacional convidado na perspectiva das syscalls de aplicação.

Em síntese, o KVM com VT-x e virtio realiza uma virtualização eficiente: hardware-transparent para CPU, com overhead mínimo para código de aplicação, e paravirtualizada para I/O, com otimizações de coalescing que, neste ambiente, resultaram em throughput comparável ao acesso direto ao disco.

Referências

- POPEK, G. J.; GOLDBERG, R. P. Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, v. 17, n. 7, p. 412–421, 1974.
- BARHAM, P. et al. Xen and the art of virtualization. *ACM SIGOPS Operating Systems Review*, v. 37, n. 5, p. 164–177, 2003.
- KIVITY, A. et al. KVM: the Linux virtual machine monitor. *Proceedings of the Linux Symposium*, v. 1, p. 225–230, 2007.
- RUSSELL, R. virtio: towards a de-facto standard for virtual I/O devices. *ACM SIGOPS Operating Systems Review*, v. 42, n. 5, p. 95–103, 2008.
- TANENBAUM, A. S.; BOS, H. *Modern Operating Systems*. 4. ed. Pearson, 2014.
- SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Operating System Concepts*. 10. ed. Wiley, 2018.